



Amazon Q Developer & AI SDLC Orchestration Skills

Day 1 of 2 – From Code Generation to Agent Orchestration

A hands-on technical training for Java & Spring Boot developers ready to move beyond code suggestions and start engineering AI-driven development systems.

DAY 1 OF 2

[HANDS-ON TRAINING](#)

Why Are We Here?

Endless Re-Explanation

You re-explain your architecture standards to AI every single session — from injection patterns to naming conventions. Every. Single. Time.

Unknown Compliance

AI suggests code and you have no reliable way to know whether it follows your team's established patterns, naming rules, or approved libraries.

Late-Stage Gaps

Security vulnerabilities and missing test coverage are caught during code review — not at the source where they're cheapest to fix.

 **Today we fix all three.** Systematically. With repeatable, team-wide solutions.



This Is Not a Prompting Course

What We're NOT Doing

- Writing cleverer prompts to get faster suggestions
- Copy-pasting AI-generated code into your IDE
- Learning how to ask AI better questions
- Speed-running code generation tasks

What We ARE Doing

- Designing agents with explicit purpose and bounded scope
- Giving agents persistent, committed knowledge of your codebase
- Automating quality gates across your entire pipeline
- Engineering a repeatable, team-wide AI system

 **The shift:** You are not a better prompt writer after today. You are an AI system designer.

Housekeeping & Setup



Today's Flow at a Glance

01

Section 1

Welcome, context, and the 5-Layer Framework mental model

02

Section 2

Q Developer setup, Labs 1–2, Steering Files

03

Section 3

Governance, Spec-Driven Development, Lab 3

04

Section 4

Security, Testing Agents, Lab 4, Capstone 1



Housekeeping

- Breaks scheduled between each module
- Repo link shared in chat — clone it now
- Wi-Fi credentials on the whiteboard
- Questions welcome during labs and transitions
- Pair up if you prefer — labs support solo or pair work

Pre-Flight Check

Before we begin, verify your environment is fully ready. Every item below must be green. If anything is missing, raise your hand **right now** — we fix it before the first lab.

1

AWS Account Active

Log in to the AWS Console and confirm your account is accessible

2

Q Developer in IDE

Amazon Q Developer extension installed and visible in your IDE sidebar

3

Q Developer Authenticated

Q icon visible in the status bar — not greyed out, not showing an error

4

Java 17+ Installed

Run `java --version` in your terminal — confirm 17 or higher

5

Maven or Gradle Ready

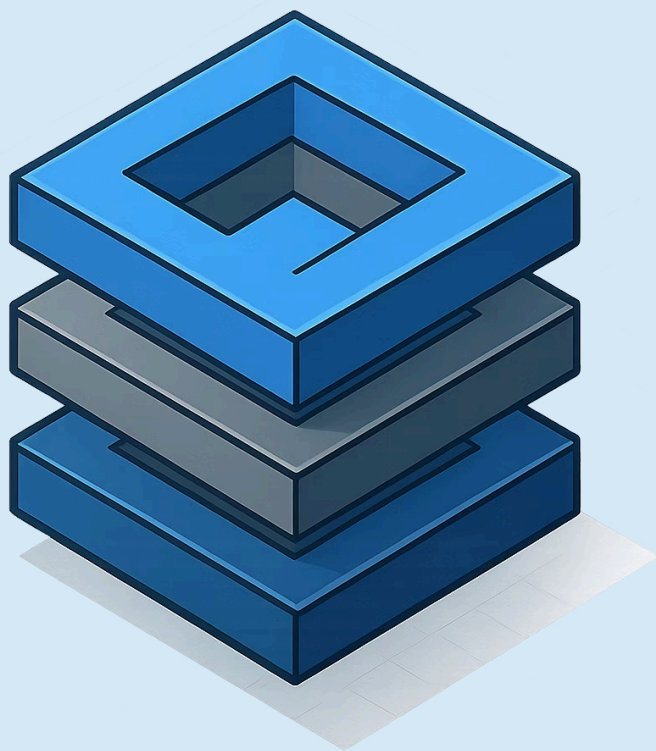
Run `mvn -v` or `gradle -v` — confirm the build tool responds

6

Git Configured & Repo Ready

Run `git config --list` — confirm user.name and user.email are set, repo cloned

 If anything is red — raise your hand now. Do not wait until the first lab.



FRAMEWORK

The 5-Layer AI SDLC Orchestration Framework

The mental model for everything we build today — and tomorrow. Every tool, every lab, every agent maps to one of these five layers.

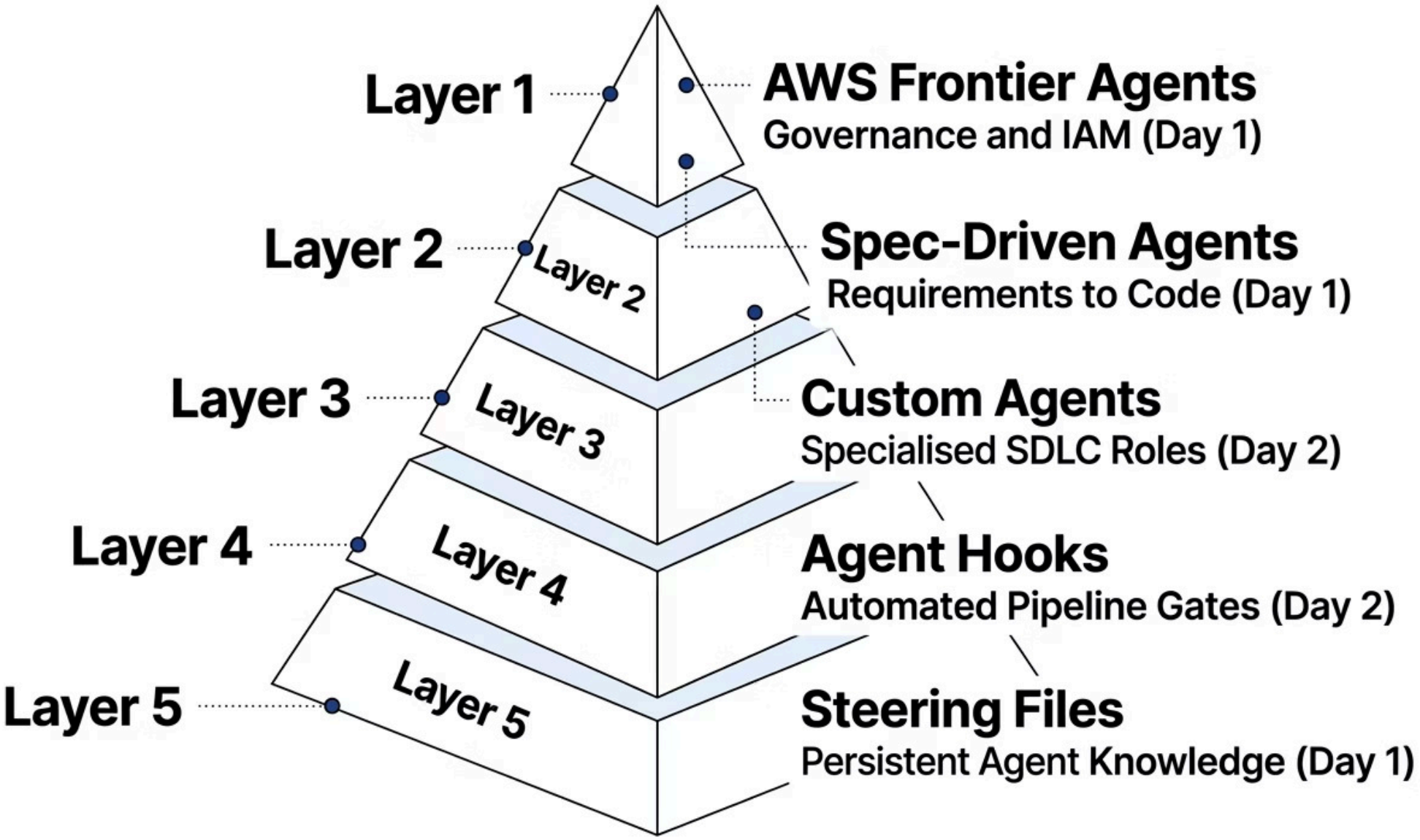
From Consumer to Designer

This is the core mindset shift. Before today, you were a consumer of AI suggestions. After today, you are the designer of AI systems.

 Before	 After
Accept whatever code AI suggests	Design agents with specific purpose and scope
Re-explain your standards every session	Steering files give AI persistent project memory
React to what AI produces	Define hooks that automate quality gates
One feature at a time, from scratch	Specs generate full requirements + architecture + tasks
Hope AI follows your patterns	Governance bounds what agents can and cannot do

 **You are not asking AI for help. You are engineering the system.**

The 5 Layers – Overview



Each layer builds on the one below it. Layer 5 (Steering Files) is the foundation – without it, every layer above starts blind. Today we build Layers 1, 2, and 5. Day 2 completes Layers 3 and 4.

LAYER 5 • FOUNDATION

Steering Files

Markdown files committed to `.kiro/steering/` in your repository. Every agent reads them automatically — no prompting required.

- Written once by your team
- Read by every agent, every session
- Committed to Git = shared team memory

Why This Is the Foundation

Every other layer depends on agents knowing your standards. Without steering files, agents are stateless — they forget everything between sessions.

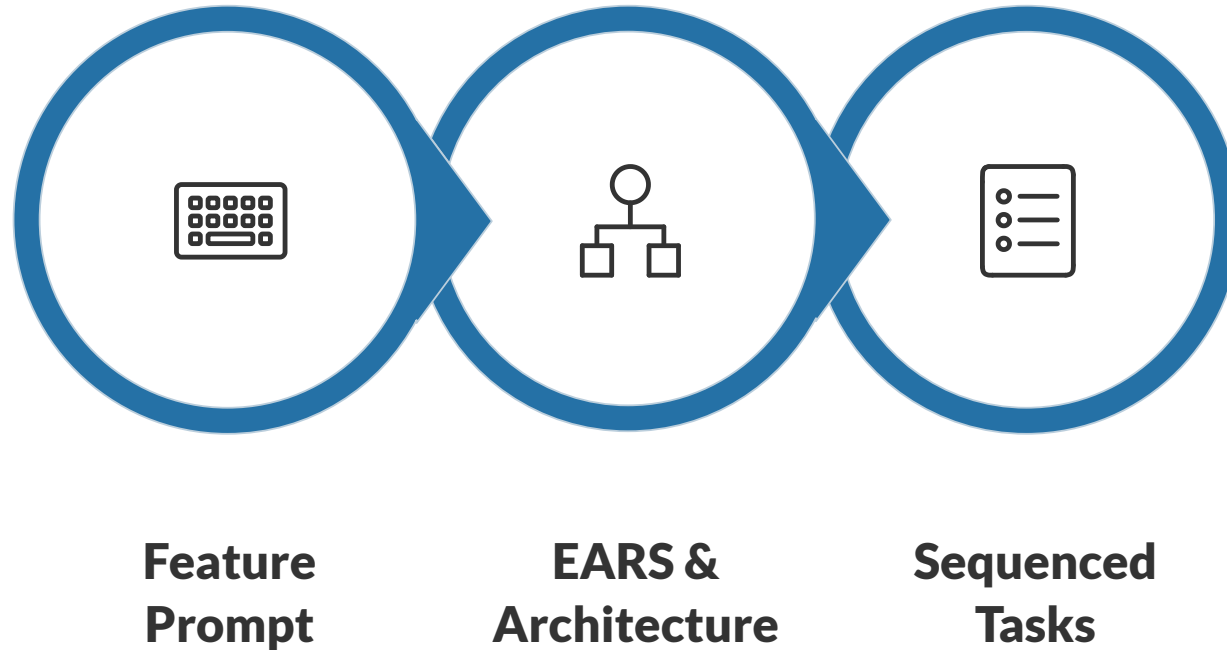
"Without steering files, every agent starts blind. With them, every agent starts expert."



Built in **Lab 2** today

Layer 2: Spec-Driven Development

Instead of typing a feature and hoping for the best, spec-driven development transforms a single natural-language prompt into a complete, reviewable blueprint before a single line of code is written.



❏ **Key insight:** One prompt generates complete requirements, architecture diagrams, and an implementation task plan — with your approval at each stage. Used in **Lab 3** today.

Layer 1: The Three Frontier Agents



Autonomous Developer Agent

Reads and writes files, runs tests, opens pull requests, and persists context across sessions. The primary agent for feature development.



DevOps Agent

Queries CloudWatch logs, interacts with CI/CD pipelines, and interrogates AWS service state. Built for infrastructure and deployment tasks.



Security Agent

Performs OWASP scanning, surfaces inline fix suggestions, and checks dependency vulnerabilities before code reaches review.



These agents have real AWS permissions. Governance is not optional. Configured in Workshop 1 today.

Layer 3: Custom Agents – Specialists, Not Generalists

Custom agents live in `.kiro/agents/` and are scoped to a single, well-defined job. Each one reads your steering files automatically, so it already knows your standards.

Code Review Agent

Permissions: `read_file` only • Model: Opus
Reviews PRs against your architecture standards without writing any files.

Test Generator Agent

Permissions: `read + write + shell` • Model: Sonnet
Generates unit and integration tests following your `testing-standards.md` exactly.

Docs Agent

Permissions: `read + write` • Model: Sonnet
Keeps OpenAPI specs and README files in sync with your actual implementation.

 Built in Lab 5 – Day 2

Layer 4: Hooks — From On-Demand to Automated

Hook Triggers → Agent Actions

→ **onSave**

Automatically generates Javadoc for any modified method

→ **onCommit**

Runs security lint and blocks the commit if critical issues are found

→ **onPush**

Syncs documentation changes to match the latest committed code

Two Operating Modes

Autopilot

Agent acts immediately on trigger. Zero human intervention required. Maximum throughput.

Supervised

Agent shows a plan and waits for approval before taking action. Maximum oversight.

 Hooks transform agents from tools you use into gates that run automatically. Built in **Lab 6 — Day 2**.



What We Build Today

Today's labs focus on **Layers 1, 2, and 5** — the governance, knowledge, and specification layers that form the foundation of every AI-driven workflow.



Layer 5 Deliverable

3 steering files authored and committed to Git — architecture, testing standards, API design



Layer 1 Deliverable

IAM policy with explicit deny block + branch protection ruleset — agent governance document saved



Layer 2 Deliverable

Complete inventory service spec — EARS user stories, architecture artefacts, approved task plan

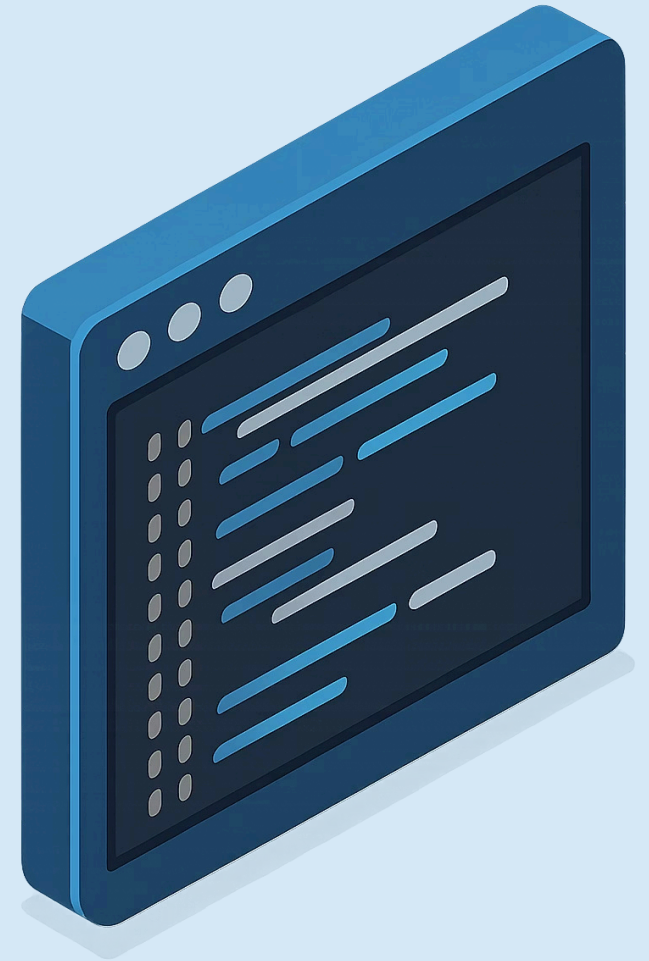


Plus: A working Spring Boot REST API via **Capstone 1**

MODULE 1.2

Amazon Q Developer

Setup & Agentic Capabilities • Lab 1



Know Which Mode You're In

Inline Suggestions — Passive

- Triggered automatically as you type
- Single-line or block-level completions
- No file read or write access
- Always on — no invocation required
- Stateless — no memory between lines

Best for: autocomplete, boilerplate, method signatures

/dev Agent Mode — Active

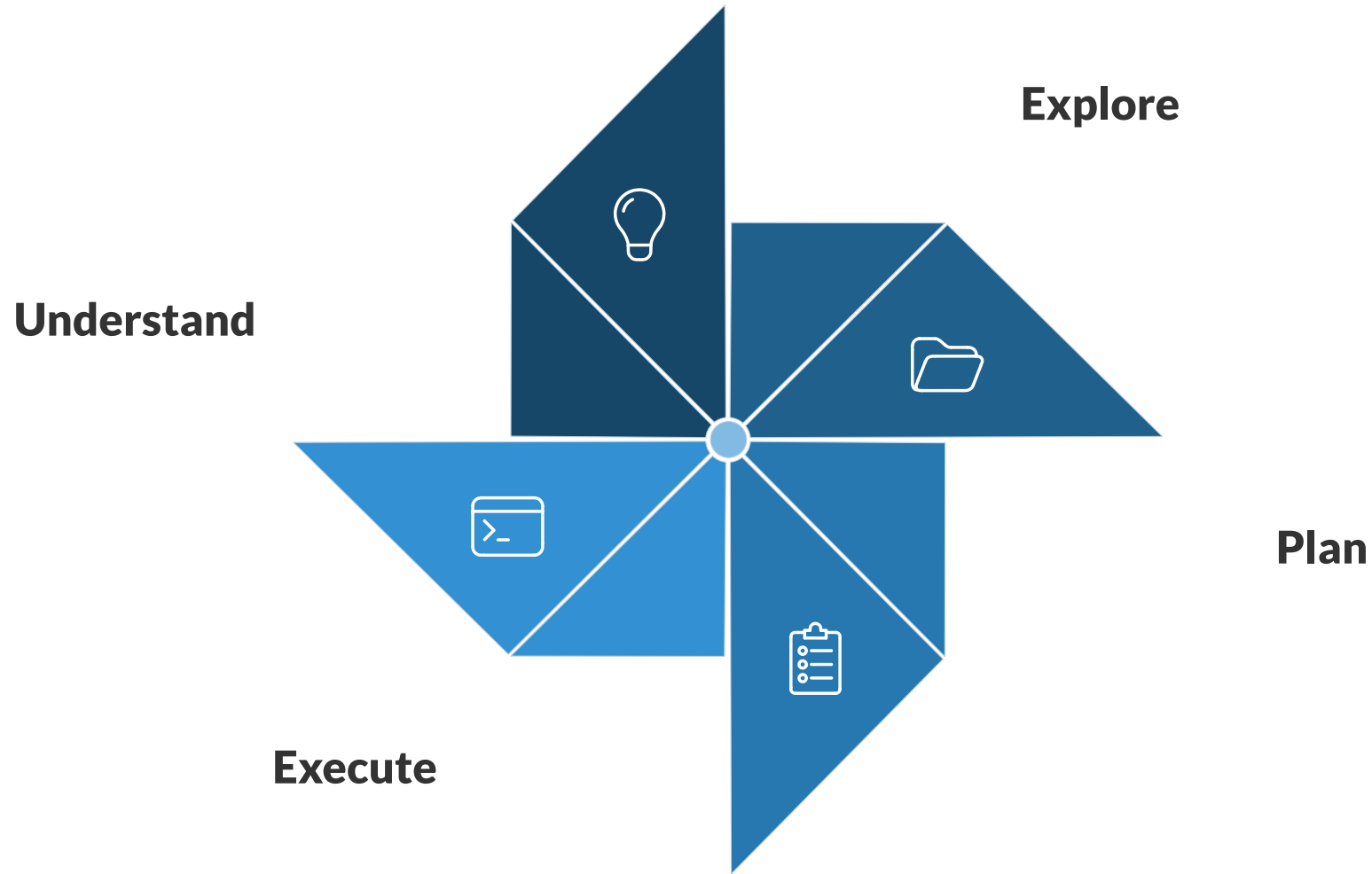
- Explicitly invoked with `/dev` command
- Reads your entire project structure
- Writes multiple files in sequence
- Can run commands and tests
- Shows a plan before acting — you review it

Best for: features, refactoring, test generation, migrations

 Knowing which mode is active determines what context the agent has. **Inline mode is never aware of your steering files.**

How /dev Works – The 5-Step Cycle

Every /dev invocation follows the same plan-execute cycle. Understanding each step tells you exactly where to intervene.



⊗ **Step 3 – PLAN is where you intervene.** Always read the full plan before accepting. Reject it if it proposes field injection, uses RestTemplate, or skips tests. This is your last cheap correction point.

Lab 1: Your First /dev Invocation

LAB 1

Use the exact prompt below in Q Developer `/dev` mode. Do not modify it — the prompt is intentionally under-specified so you can observe what the agent does with gaps in context.

```
/dev Implement a user registration feature.  
POST /api/v1/users/register  
Accept: email, password, firstName, lastName  
Validate all fields.  
Hash password with BCrypt.  
Return 201 with userId and email.  
Return 409 if email exists.  
Use constructor injection.  
No RestTemplate.
```

Watch For These Four Things

Does the **plan** match your expectations before you accept?

Is **constructor injection** used — not field injection?

Is **BCrypt** used for password hashing?

Are **tests generated** automatically or skipped?

Review, Don't Accept Blindly

The most common agent mistake in Java/Spring Boot projects is defaulting to field injection. Here's how to spot it — and why it matters.

✗ Field Injection — Reject This

```
@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private PasswordEncoder encoder;
}
```

Mutable, untestable, hides dependencies. Violates your architecture standards.

✓ Constructor Injection — Accept This

```
@Service
public class UserService {

    private final UserRepository userRepository;
    private final PasswordEncoder encoder;

    public UserService(
        UserRepository userRepository,
        PasswordEncoder encoder) {
        this.userRepository = userRepository;
        this.encoder = encoder;
    }
}
```

Immutable, testable, explicit. Correct.

💡 Steering files (Lab 2) will prevent the agent from ever suggesting field injection again. But for now — you catch it manually.

Lab 1 – Debrief

LAB 1 • DEBRIEF

Before we move on, take 5 minutes to reflect on what the agent produced. These four questions are the foundation for understanding why steering files exist.

“

"What did the agent get right on the first attempt — without any guidance?"

”

“

"What did you need to correct or regenerate — and why?"

”

“

"Did the agent follow your architectural standards without being told?"

”

“

"How long would this feature have taken without the agent — and how much review did you need to do?"

”

❏ The corrections you just made are exactly what Lab 2 automates. Every manual fix you did becomes a line in a steering file.



MODULE 1.3

Governing Frontier Agents

Layer 1 • Workshop 1

Autonomous Agents With Real Permissions



Autonomous Developer

Reads and writes files, runs test suites, opens pull requests, and persists context across multiple sessions.



DevOps Agent

Queries CloudWatch logs, interacts with CI/CD pipelines, interrogates AWS service configuration and state.



Security Agent

Performs OWASP scanning, surfaces inline fix suggestions, and checks dependency vulnerability databases.



These are not chat assistants. They take real actions with real consequences. Governance is not optional — it's the first thing you configure.



Design Agent IAM Like You Design Human IAM

The minimum privilege principle applies to agents exactly as it applies to human users. If an agent doesn't need a permission to do its job, it should not have it.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadPermissions",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "cloudwatch:GetMetricData",
        "logs:DescribeLogGroups"],
      "Resource": "*"
    },
    {
      "Sid": "WriteFeatureBranchesOnly",
      "Effect": "Allow",
      "Action": ["codecommit:GitPush"],
      "Resource": "arn:aws:codecommit:*:*:*",
      "Condition": {
        "StringLike": {
          "codecommit:References": ["refs/heads/feature/*",
            "refs/heads/kiro/*"]
        }
      }
    },
    {
      "Sid": "ExplicitDeny",
      "Effect": "Deny",
      "Action": ["iam:*", "s3:DeleteBucket",
        "rds:DeleteDBInstance", "organizations:*"],
      "Resource": "*"
    }
  ]
}
```

✗ **Always include an explicit DENY block.** An Allow-only policy is incomplete — a Deny block ensures safety even if permissions drift.

The Non-Negotiable Deny List

These six permissions must always appear in your explicit DENY block. No exceptions — regardless of what the agent needs to accomplish.

iam:*

Privilege escalation risk. An agent that can modify IAM can grant itself any permission.

Push to main

Bypasses human review entirely. All agent commits must go through a PR.

s3:DeleteBucket

Irreversible infrastructure destruction. There is no undo for a deleted S3 bucket.

rds:DeleteDBInstance

Irreversible data loss. Even with backups, recovery is measured in hours.

Production Secrets

Credential exposure risk. Agents should never have access to production secret stores.

organizations:*

Org-level configuration changes. An agent must never touch your account hierarchy.

The Agent Cannot Merge to Main

Git Branch Strategy

main

🔒 Protected — human PR required. Agent IAM role NOT in push allowlist.

feature/agent-*

✅ Agent allowed to push and commit freely.

kiro/*

✅ Agent allowed — Kiro-managed branches for spec and steering work.

Required Branch Protection Settings

- Required PR reviews: **minimum 1 human reviewer**
- Dismiss stale reviews on new push: **ON**
- Required status checks: `ci/tests`, `ci/security-scan`
- Agent IAM role: **NOT in push allowlist for main**
- Force push to main: **DISABLED**

❌ If the agent can push directly to main, your governance document is incomplete.

Choose Your Network Security Perimeter

Tier 1 — Internet Open

Use for: Personal prototyping only

Access: Any public internet endpoint

Risk: Medium — data may leave your network boundary

Tier 2 — Restricted Egress

Use for: Standard development teams

Access: Allowlisted hosts only

Risk: Low — outbound traffic explicitly controlled

Tier 3 — Fully Isolated

Use for: Financial, healthcare, government

Access: Internal network only — no public internet

Risk: Minimal — fully air-gapped agent operation

 **Decision rule:** Choose your tier based on data residency requirements and your organisation's security policy — not based on convenience.

Workshop 1: Design Your Team's Governance

WORKSHOP 1

Your deliverable from this workshop is a governance document saved to `.kiro/governance/agent-governance.md`. This document defines the rules every agent must operate within on your team.

1

IAM Policy

Write the full IAM policy JSON including ReadPermissions, WriteFeatureBranchesOnly, and an explicit ExplicitDeny block

2

Branch Protection Rules

Document your branch strategy — which branches agents can write to, which are human-only, and the required status checks

3

Network Access Tier

Select Tier 1, 2, or 3 and write a one-paragraph rationale based on your data residency requirements

4

Approval Matrix

Define which agent actions require human approval before execution — and which may run in autopilot mode

MODULE 1.4

Authoring Steering Files

Layer 5 — The Foundation • Lab 2



Why Doesn't the Agent Know Your Standards?

Repeating Instructions

Developer teaches standards in each chat, agent forgets between sessions.



Steering Files

Write standards once; every agent reads them in every session.



Write once. Every agent reads it. Every session. Every developer on your team. Steering files are your team's institutional memory — committed to Git and version-controlled like any other code.

A Steering File Is Just Markdown

Steering files have a simple two-part structure: a YAML front matter header that controls loading behaviour, and a Markdown body that contains your standards.

```
---
description: Core technology stack and architectural standards
alwaysApply: true
---

# Architecture Standards

## Stack
Spring Boot 3.x, Java 17, PostgreSQL, SQS

## Package Structure
com.example.{service}.{domain | application | infrastructure | api}

## Approved Libraries
- Spring Data JPA (repositories)
- WebClient (HTTP — never RestTemplate)
- Testcontainers (integration tests — never H2)

## Forbidden Patterns
- RestTemplate: use WebClient instead
- Field injection (@Autowired on field): use constructor injection
- H2 in tests: use Testcontainers
```

Three sections: **YAML front matter** (loading control) • **Description** (agent context) • **Body** (the actual standards)

Control When Each File Loads

Not every steering file should load for every context. Token efficiency matters — give the agent what it needs, when it needs it.

Always On

`alwaysApply: true`

Core architecture standards, security rules, naming conventions. These apply to every file, every session, every agent.

File Pattern

`filePattern: "*.java"`

Stack-specific rules loaded only when the agent is editing Java files. Keeps context focused for other file types.

On Demand

`alwaysApply: false`

Troubleshooting guides, rarely-needed reference material. Loaded only when explicitly requested by the agent or developer.

 **Design principle:** Keep context focused and token-efficient. An agent overloaded with irrelevant context makes worse decisions.

File 1: architecture.md

STEERING FILE 1 OF 3

What It Contains

Tech Stack: Spring Boot 3.x, Java 17, PostgreSQL, SQS

Package Structure: domain / application / infrastructure / api

Approved Libraries Table: Spring Data JPA, WebClient, Testcontainers

Forbidden Patterns: RestTemplate → WebClient, field injection → constructor injection

Load Strategy

`alwaysApply: true`

This file must load for every single agent invocation. It is the source of truth for all structural decisions.

Impact

Without this file, the agent in Lab 1 defaulted to `@Autowired` field injection and could have used RestTemplate. With this file, those mistakes become impossible.

File 2: testing-standards.md

STEERING FILE 2 OF 3

Naming Convention

All test methods must follow:

`should{Behaviour}_when{Condition}`

Example: `shouldReturn409_whenEmailAlreadyExists`

Coverage Requirements

80% minimum line coverage. Every service method must have at least 1 happy path test and 1 failure path test.

AAA Structure

Every test must follow Arrange / Act / Assert with explicit comments marking each section. No exceptions.

Tooling Rule

Integration tests: **Testcontainers ALWAYS.**

H2 in-memory database: **NEVER.**

Violations are a hard rejection in code review.

File 3: api-design.md

STEERING FILE 3 OF 3

URL Conventions

- Plural nouns: /users, /inventories
- Version prefix: /api/v1/
- Lowercase, hyphen-separated: /stock-reservations
- No verbs in paths: /users not /getUsers

HTTP Status Codes

- 201 Created → successful POST
- 204 No Content → successful DELETE
- 409 Conflict → duplicate resource
- 404 Not Found → resource missing

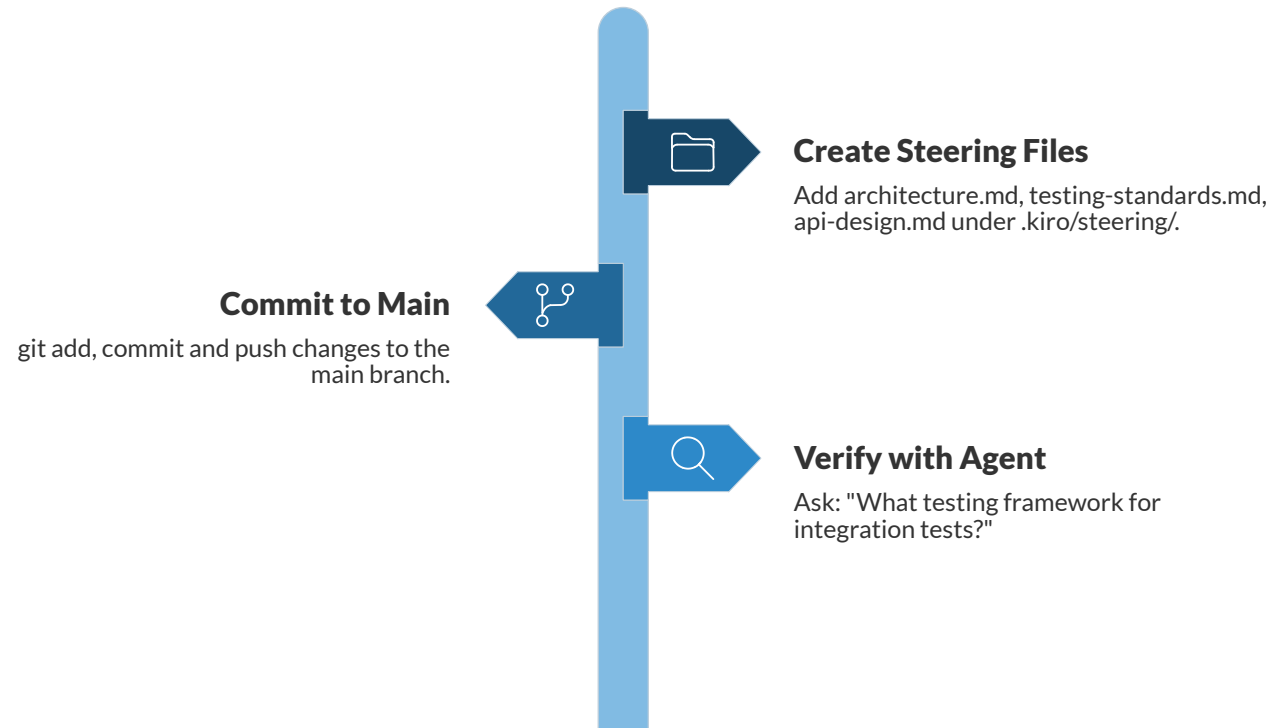
Standard Error Format

```
{
  "timestamp": "2024-01-15T10:30:00Z",
  "status": 409,
  "error": "Conflict",
  "message": "Email already exists",
  "path": "/api/v1/users/register",
  "traceId": "abc-123-def"
}
```

Every error response — 4xx and 5xx — must match this exact structure. The `traceId` field is mandatory for production debugging.

Lab 2: Write, Commit, Verify

LAB 2



✓ **Verification test:** Ask the agent: *"What testing framework should I use for integration tests?"*

✓ Expected answer: **"Testcontainers"**

✗ If it says H2 — your steering files are not loading. Check the YAML front matter.

Completion criteria: 3 files committed to .kiro/steering/, 2 verification questions answered correctly by the agent with no prompting.



MODULE 1.5

Spec-Driven Development

Layer 2 • Lab 3

One Prompt. Complete Requirements + Architecture + Tasks.

15

Backend APIs

Built by one team in 3 days using the spec workflow — a task that typically takes 2–3 weeks of planning and development

3

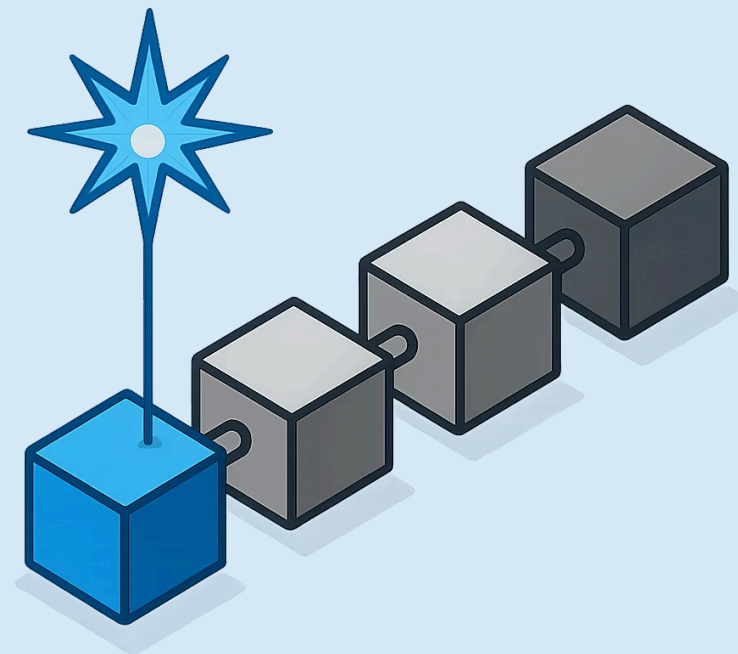
Approval Gates

Requirements, Architecture, and Task Plan — each reviewed and approved by a human before implementation begins

0

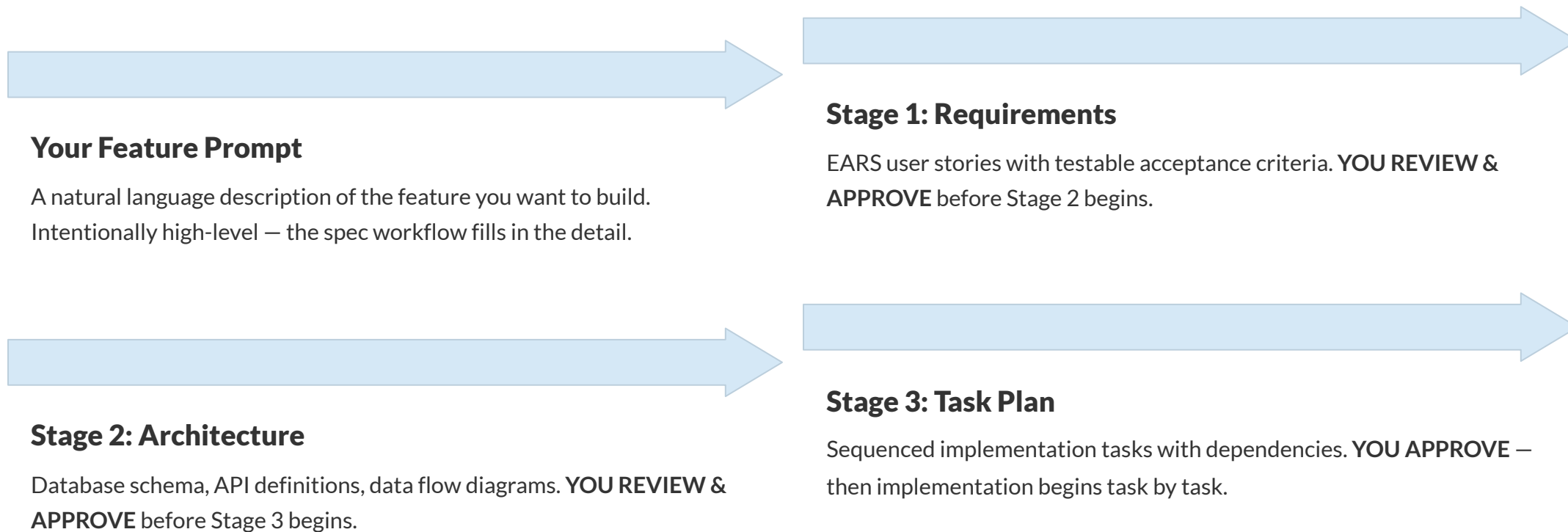
Re-Explanations

The spec becomes the source of truth — no re-explaining context, standards, or architecture decisions to the agent



The Spec Workflow – 3 Approval Gates

You are the reviewer at every gate. The agent generates — you approve or request changes. Nothing proceeds without your explicit sign-off.



❏ **Your role is reviewer, not generator.** The agent writes everything. You decide what's right.

Stage 1: Requirements in EARS Format

EARS (Easy Approach to Requirements Syntax) produces requirements that are testable by definition. Each user story directly maps to one or more acceptance test cases.

EARS Syntax

WHEN [event/trigger]
THE SYSTEM SHALL [required behaviour]
IF [optional precondition]

The structure forces specificity. Vague requirements cannot be written in EARS format — which is the point.

Example User Stories

US-002: Stock Reservation

WHEN a reservation request is received, IF sufficient stock exists, THE SYSTEM SHALL reserve the requested quantity and return 201.

US-003: Insufficient Stock

WHEN a reservation request is received, IF stock is below the requested quantity, THE SYSTEM SHALL reject the request and return 409.



EARS stories are testable by definition. Use them directly as acceptance criteria in your test suite.

Don't Just Accept — Review Stage 1

The agent will generate a solid first draft. But it will miss edge cases. Your job is to find the gaps before they become bugs in production.

Concurrent Reservation

Is the concurrent reservation scenario covered? Two requests for the last unit arriving simultaneously — what happens?

Edge Case Coverage

Are all edge cases captured? Product not found (404), stock already released (409), negative quantities (400)?

Testability of Criteria

Are the acceptance criteria testable and specific — or vague and unmeasurable? "System responds appropriately" is not acceptable.

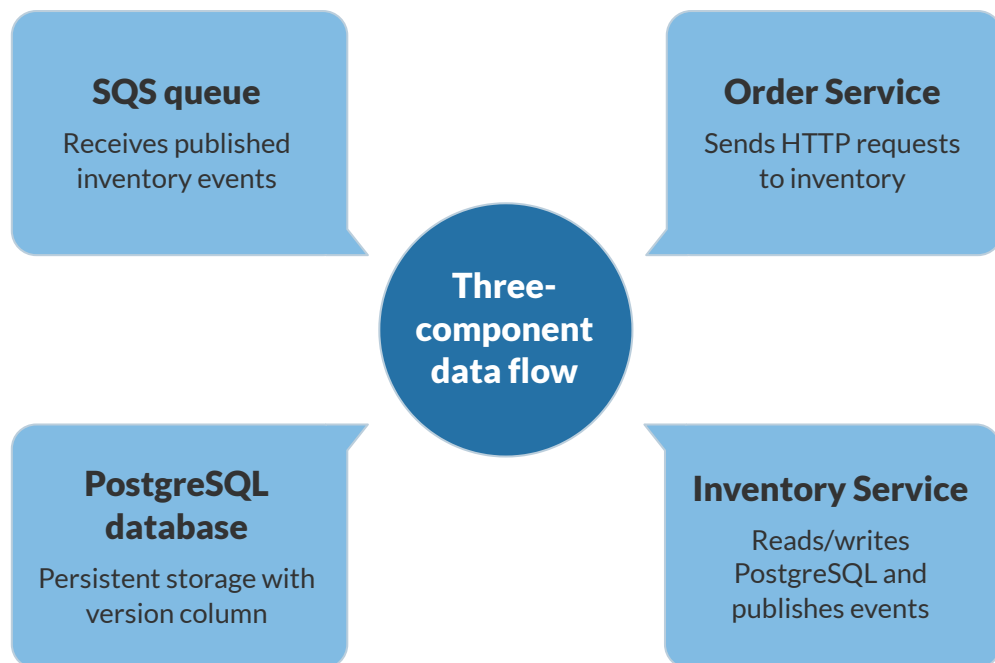
Business Rule Accuracy

Are business rules correctly reflected? A release of a reservation that was never made — what should happen?

 **Add US-009: Concurrent Reservation Safety** before approving Stage 1. The agent almost never generates this without prompting.

Stage 2: What the Architecture Artefacts Include

Data Flow Diagram



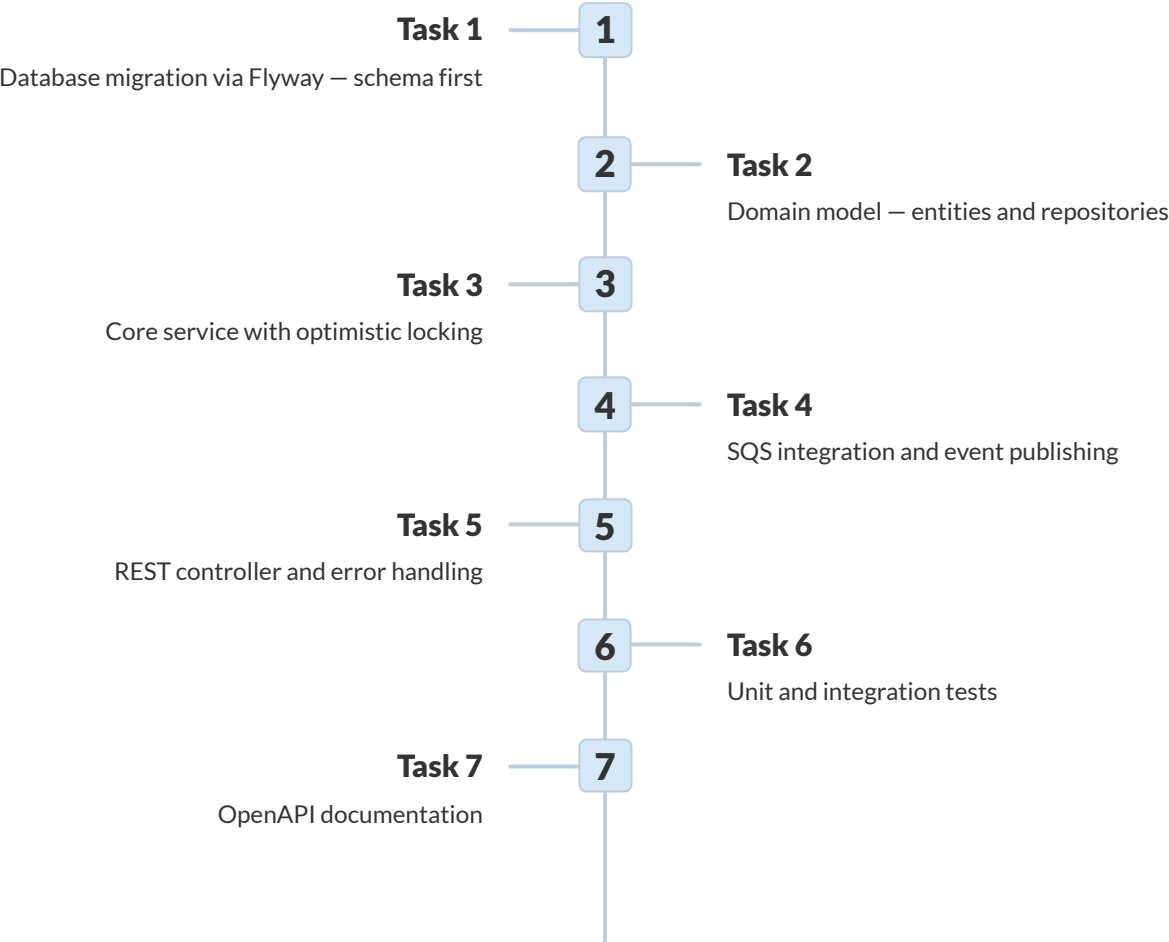
Review Checklist for Stage 2

- Is a `version` column present in `inventory_items`? (Required for optimistic locking)
- Are HTTP status codes correct? (201 creates, 409 conflicts, 404 not found)
- Does the error response format match `api-design.md`?
- Is the `StockChangedEvent` schema defined for SQS?
- Are all 3 API endpoints explicitly defined with methods and paths?

❌ Missing the `version` column is the most common Stage 2 miss. If it's absent, request it before approving.

Stage 3: Your Implementation Roadmap

Stage 3 produces a sequenced task plan where each task depends on the completion of the task before it. The order is not arbitrary — it reflects the dependency chain of your application.



Implementation Is Now Sequential

With an approved spec and task plan, each `/dev` invocation is focused, bounded, and unambiguous. The spec is the source of truth — no re-explaining, no guesswork.

`/dev Implement Task 1: Create the Flyway migration`
`for` the inventory schema as defined `in` the spec.

Tables required:

- inventory_items (`id`, sku, quantity, version, created_at)
- reservations (`id`, inventory_item_id, quantity, status, created_at)

Use sequential versioning: `V1__create_inventory_schema.sql`

- Review the generated migration — verify column types, constraints, and the `version` column is present
- Run `./mvnw flyway:migrate` to verify the migration applies cleanly
- Invoke `/dev Task 2` only after Task 1 is verified — never skip ahead

❏ **No ambiguity. No re-explaining. The spec is the source of truth.**

Lab 3: Write Your First Spec

LAB 3

Use the inventory management service as your feature. The prompt below starts the spec workflow — the agent generates Stage 1 first and waits for your review.

`/spec Create` an inventory management service.

The service manages stock levels `for` an e-commerce platform.

Core operations:

- Reserve stock `for` an `order` (POST)
- `Release` a reservation (`DELETE`)
- Query `current` stock level (`GET`)

Constraints:

- Concurrent reservation safety required
- Events published `to` SQS `on` stock change
- PostgreSQL `with` optimistic locking

Add These If the Agent Misses Them in Stage 1

US-007: Unknown Product — system returns 404 when SKU does not exist

US-008: Idempotent Release — releasing a non-existent reservation returns 409, not 500

US-009: Concurrent Reservation Safety — optimistic locking prevents overselling under concurrent load

✓ Output: Spec committed to `.kiro/specs/` — this becomes the source of truth for Capstone 1

MODULE 1.6

Security, Testing & Review Agents

Lab 4



The /scan Agent – Security at the Source

Run `/scan` in Q Developer chat to perform a static security analysis of your codebase. Issues are categorised by severity and surfaced with inline fix suggestions.

SQL Injection

Unparameterised queries with user-controlled input — one of the most critical and common Java vulnerabilities.

Hardcoded Credentials

API keys, passwords, or secrets committed directly in source code or configuration files.

Insecure Random

`java.util.Random` used for token generation or security-sensitive operations — predictable output.

XXE Vulnerabilities

XML parsers configured to process external entity references — allows file system access or SSRF.

Disabled SSL

`setSSLSocketFactory` or `hostnameVerifier` configured to accept all certificates or skip validation.

Unvalidated Redirects

Redirect targets derived from user input without allowlist validation — open redirect vulnerability.

✅ Catch these at commit time, not in production. Invoke: `/scan` in Q Developer chat.

Generate Tests From a Single Command

The `/test` agent reads your service class, understands its behaviour, and generates a complete test suite — following your steering file standards automatically.

What `/test` Generates

- **Unit tests** for service classes — Mockito for all dependencies
- **Integration tests** for repositories — Testcontainers, never H2
- **MockMvc tests** for controllers — request/response cycle with error format verification

Invoke It

```
/test InventoryService
```

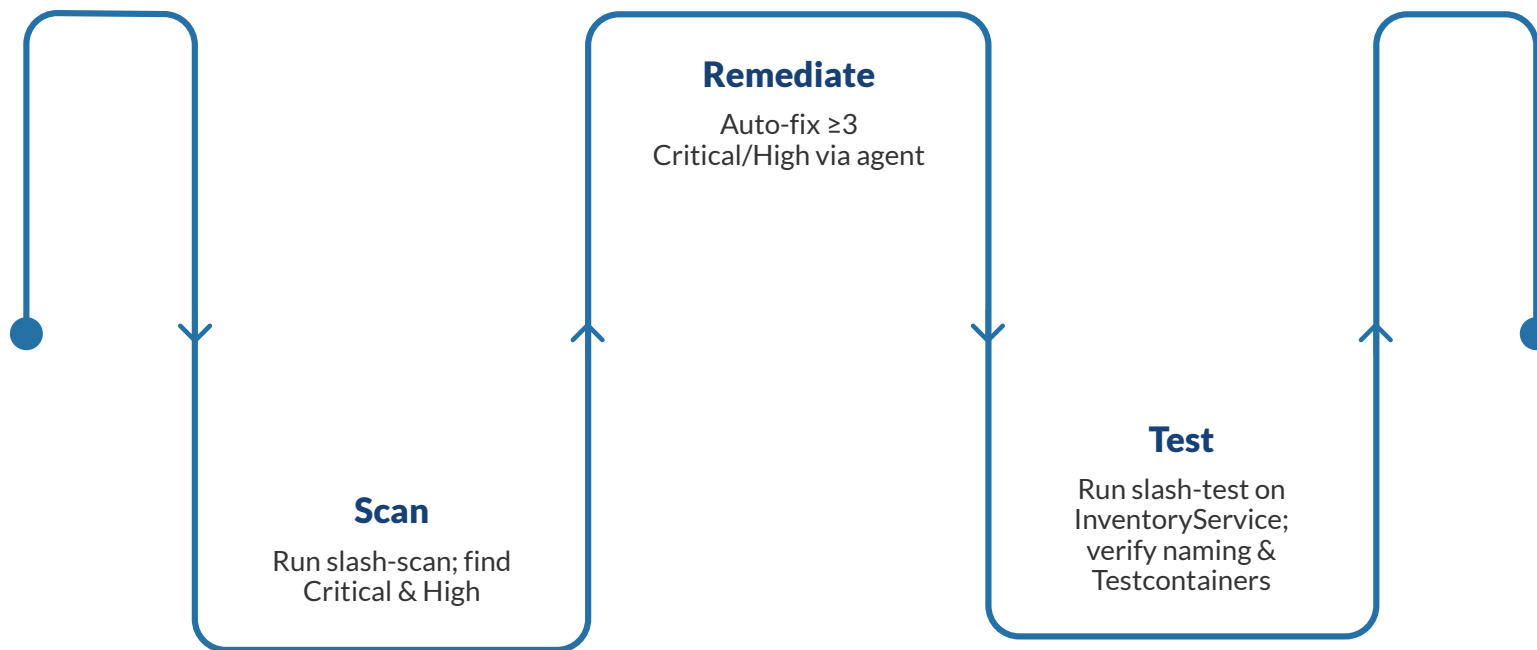
Automatic Standards Compliance

Because the agent reads your steering files before generating, every test it produces will:

- Follow the `should{Behaviour}_when{Condition}` naming convention
- Use the Arrange / Act / Assert structure with section comments
- Use Testcontainers for any database interaction
- Include at minimum 1 happy path and 1 failure path per method

Lab 4: Scan, Fix, Generate

LAB 4



Success Criteria

Security Scan

Zero Critical issues remaining after remediation. High issues documented with mitigation plan.

Test Quality

Generated tests follow naming convention, AAA structure, and use Testcontainers — not H2.

Tests Pass

Run `./mvnw test` — all generated tests pass on first run without manual fixes.

How Layers 1, 2 & 5 Work Together



Layer 5: Steering Files

The agent knows your stack, patterns, and forbidden anti-patterns before you type a single word. Context is pre-loaded from Git.



Layer 2: Spec Workflow

Your feature prompt generates a complete, reviewable plan — EARS requirements, architecture artefacts, and a sequenced task list — before code is written.



Layer 1: Governance

The agent acts within your defined IAM boundaries and branch protection rules. It cannot merge to main, delete infrastructure, or escalate privileges.



Result: Every `/dev` invocation is informed by your standards, bounded by your governance, and fully reviewable at every step.



What Day 2 Adds: Layers 3 & 4

Today's foundation — Layers 1, 2, and 5 — makes agents knowledgeable and governed. Day 2 makes them specialised and automated.

Layer 3 — Lab 5

Custom Agents in

CAPSTONE 1

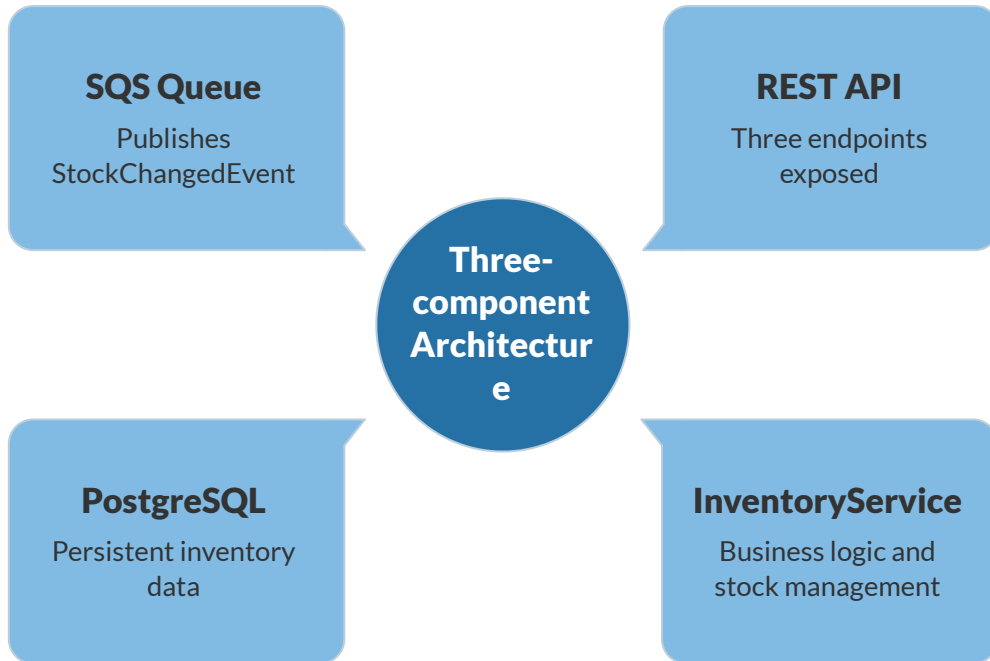
Spring Boot Inventory REST API

Apply Layers 1, 2 & 5 together — build a production-grade service from spec to working endpoint



What You're Building in Capstone 1

Architecture



Success Criteria

- ✓ Application starts without errors
- ✓ At least 2 endpoints return correct responses
- ✓ `./mvnw test` passes — all green
- ✓ `/scan` returns zero Critical issues
- ✓ Constructor injection used throughout
- ✓ Error format matches `api-design.md`
- ✓ Test names follow naming convention

Execute the Spec Task by Task

Use your approved spec from Lab 3 as the source of truth. For each task, invoke `/dev`, review the plan, verify the output, then move to the next task.

1

DB Migration

Flyway schema — tables, constraints, version column

2

Domain Model

JPA entities and Spring Data repositories

3

Core Service

Business logic with optimistic locking

4

SQS Integration

StockChangedEvent publishing

5

REST Controller

Endpoints and error handling

6

Tests

Unit + integration, Testcontainers

7

OpenAPI Docs

Swagger UI with full endpoint documentation



If time is short: Focus on Tasks 1–5 as a minimum. Tasks 6 and 7 are completed using `/test` and `/dev` in the evening or during Day 2 warm-up.

What to Show at the Demo

CAPSTONE 1 • DEMO

At the end of Capstone 1, each participant demonstrates their working service. Here's exactly what to show.

1

Swagger UI Running

Navigate to `http://localhost:8080/swagger-ui.html` — all 3 endpoints visible and documented with request/response schemas

2

Live curl Demonstration

Execute: `POST /api/v1/inventory/reservations` — show the 201 Created response with correct JSON body and `traceld` in the error format

3

Tests All Green

Run `./mvnw test` in the terminal during the demo — all tests pass, Testcontainers spins up and tears down correctly

4

Point Out One Steering File Rule

Show one concrete example: constructor injection in the service class, OR error format matching `api-design.md`, OR a test name following the convention

Day 1 Complete — What Did You Build?

✓ Q Developer Active

Authenticated in IDE, both inline and `/dev` agent modes verified and working

✓ Governance Document

IAM policy with explicit deny block + branch protection rules saved to `.kiro/governance/`

✓ 3 Steering Files

architecture.md, testing-standards.md, api-design.md — committed to Git and verified against live agent queries

✓ Inventory Service Spec

EARS user stories, approved architecture artefacts, and sequenced task plan committed to `.kiro/specs/`



Knowledge Check – Five Questions

Before we close, verify your understanding of today's core concepts. These questions span all five layers of the framework.

Q1 – Modes

What is the key operational difference between `/dev` agent mode and inline suggestions? What can each mode do that the other cannot?

Q2 – Steering Files

Why must steering files be committed to Git rather than stored locally? What happens to team-wide agent consistency if they are not?

Q3 – IAM Governance

Name two IAM permissions that must always appear in your agent's explicit DENY block – and explain why each one is non-negotiable.

Q4 – EARS Format

What does EARS stand for, and why does the EARS format produce better acceptance criteria than conventional user story formats?

Q5 – Spec Workflow

What are the three stages of the spec workflow, and what is your role at each stage? At which stage does code generation begin?

Tonight's Action: Install Kiro IDE

Why You Need It for Day 2

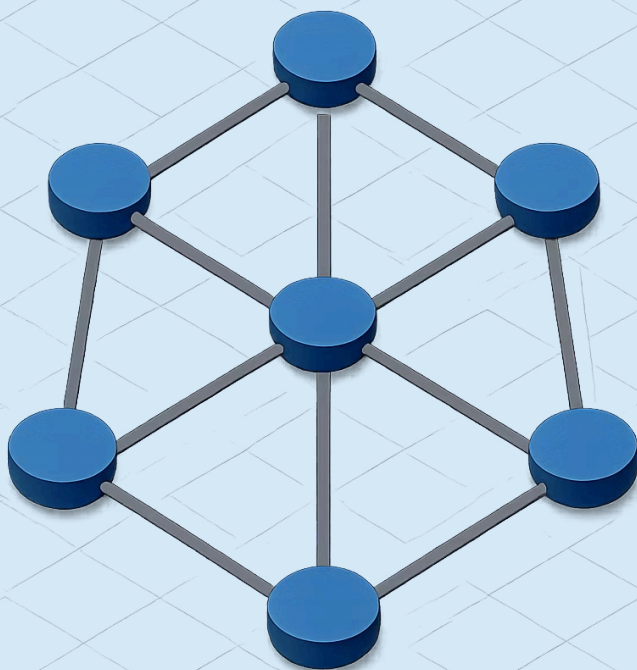
Day 2 labs build Layers 3 and 4 — Custom Agents and Hooks. Both require the Kiro IDE, which provides the `.kiro/agents/` and hook configuration interfaces that standard IDE extensions do not support.

⊗ Without Kiro IDE, you cannot complete Labs 5 or 6, or Capstone 2.

Installation Steps

1. Visit `kiro.dev/downloads`
2. Download the version for your OS (macOS / Windows / Linux)
3. Launch Kiro IDE and sign in with your AWS account
4. Verify Q Developer is active in the sidebar
5. Open your training repo — confirm `.kiro/` directory is visible

✓ Expected install time: under 10 minutes. If you hit issues, bring a screenshot to Day 2.



Day 2 Preview: What's Coming

Layer 3 – Lab 5

Custom Agents

Build purpose-scoped agents in `.kiro/agents/`: Code Review Agent (read-only, Opus), Test Generator (read+write+shell, Sonnet), Docs Agent (read+write, Sonnet)

Layer 4 – Lab 6

Hook Pipeline

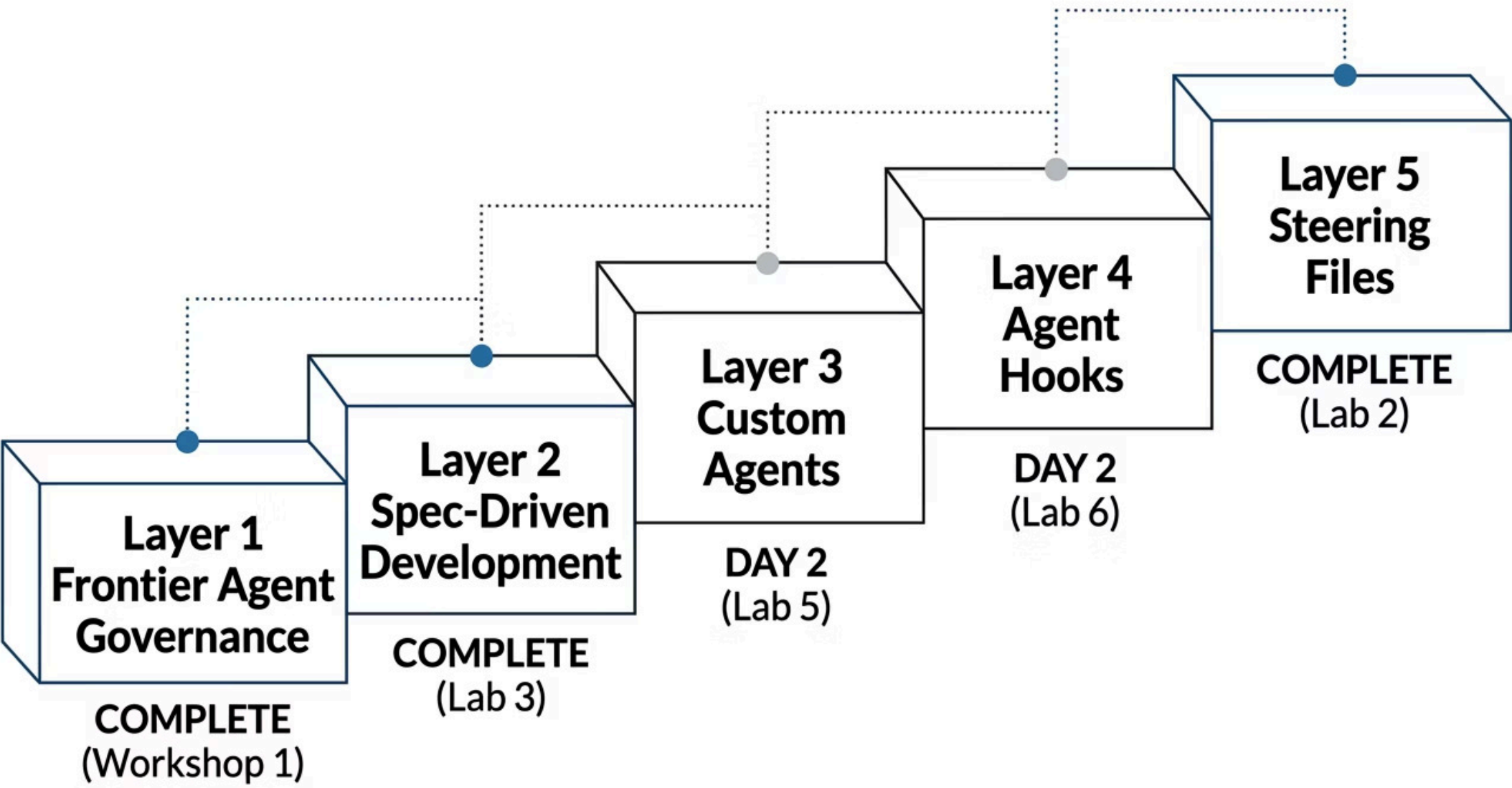
Wire agents to lifecycle triggers: `onSave` → Javadoc, `onCommit` → security lint gate, `onPush` → documentation sync. Autopilot vs supervised modes.

Capstone 2

All 5 Layers Active

Complete the inventory service with all five layers running simultaneously. Custom review agent + hook gates + spec workflow + steering files + governance boundaries.

The 5-Layer Framework — Where You Stand



Three of five layers are operational. Your agents are now **knowledgeable** (steering files), **governed** (IAM + branch protection), and **spec-driven** (approved task plan). Day 2 makes them **specialised** and **automated**.



See You on Day 2

You've moved from accepting whatever AI suggests to engineering the system that governs, informs, and guides every agent invocation. That's the shift.

✓ Tonight

Install Kiro IDE from kiro.dev/downloads. Verify it opens your training repo and Q Developer is active.

📋 Optional

Review your 3 steering files and ask: what's missing? What patterns do you enforce in PRs that aren't captured yet?

🚀 Day 2

Layers 3 & 4. Custom specialists. Automated pipeline gates. All 5 layers active in Capstone 2.



You are not asking AI for help. You are engineering the system.